

DriveSafe: Intelligent Driver Telemetry & Edge Vision System

Technical System Architecture, Computer Vision Models & Project Defense Guide

DriveSafe is an enterprise-grade IoT and computer vision driver safety platform designed to eliminate driver fatigue accidents, prevent unauthorized vehicle operation via biometric binding, enforce mandatory rest regulations, and stream real-time blackbox telemetry (GPS, speed, SOS, shift status) to a cloud command center. It bridges an ESP32 hardware edge microcontroller with an AI-driven vision server and a real-time web telemetry dashboard.

Subsystem	Tech Stack	Core Responsibilities
Edge Microcontroller	ESP32 Dev Module FreeRTOS / C++ / Arduino	RFID RC522 access control, TinyGPS++ NEO-6M telemetry, SSD1306 OLED HUD, Piezo siren, SOS interrupt, local HTTP web server, Firebase REST client.
AI Vision Backend	Python 3.13 / Flask dlib / OpenCV / face_rec	Biometric driver enrollment, 128-D facial embeddings matching, 68-point facial landmark tracking, real-time EAR drowsiness detection, ESP32 auto-dispatch.
Cloud Telemetry	Firebase Realtime DB REST API (HTTPS)	Live streaming of telemetry: GPS coordinates, speed, battery, signal, active driver metadata, emergency SOS flags, trip audit history.
Fleet Web Portal	React 18 / TypeScript Vite / TanStack Router	Live driver telemetry dashboard, pre-trip facial authentication portal (/verify), driver enrollment UI (/admin), SOS siren triggers, glassmorphic HUD.

1. Computer Vision & Machine Learning Models

DriveSafe incorporates two dedicated computer vision pipelines executing locally for low latency and zero recurring cloud inference costs:

A. Facial Recognition & Verification Model (ResNet-34 Architecture)

- **Base Model:** Built upon a deep ResNet-34 convolutional neural network trained on a dataset of approximately 3 million human faces (FaceNet / LFW benchmark accuracy of 99.38%).
- **Embeddings Representation:** When an image is captured during enrollment, the model detects facial boundaries via HOG (Histogram of Oriented Gradients) and projects the face into a dense **128-dimensional continuous vector space** (face embedding).
- **Similarity Metric:** Distance between an incoming verification frame vector v_p and enrolled vectors v_e is calculated using **Euclidean (L2) Norm**: $d = \|v_p - v_e\|_2$.
- **Classification Threshold:** Set at **0.60**. Distances ≤ 0.60 represent a positive biometric match; values > 0.60 are rejected as unauthorized.
- **Storage:** Enrolled embeddings are persisted in faces.pkl containing name, rfid_uid, encoding (128-float array), and enrolled_at.

B. Drowsiness Detection Model (Shape Predictor 68 Face Landmarks)

- **Downloaded Model File:** shape_predictor_68_face_landmarks.dat (~99.7 MB).
- **Algorithm Origin:** Bahram Kazemi and Josephine Sullivan's *Ensemble of Regression Trees (ERT)* (CVPR 2014), trained on the iBUG 300-W face dataset.
- **Inference Mechanism:** In real-time video frames (30 FPS), dlib's frontal face detector locates the bounding box, and the 68-point regressor locates exact coordinates of key facial landmarks in sub-milliseconds (eyes, eyebrows, nose, mouth, jawline).
- **Eye Landmark Indices:** Left Eye = Points [36, 37, 38, 39, 40, 41]; Right Eye = Points [42, 43, 44, 45, 46, 47].

C. Eye Aspect Ratio (EAR) Formula & Mathematical Logic

The Eye Aspect Ratio (Soukupová & Mach, 2016) Equation:

$$EAR = (\| p_2 - p_6 \| + \| p_3 - p_5 \|) / (2 \times \| p_1 - p_4 \|)$$

- **Numerator:** Sum of the two vertical eye landmark distance vectors (eyelid opening).
- **Denominator:** Twice the horizontal eye landmark distance vector (eye width).
- **Invariance Property:** EAR is scale-invariant and largely head-orientation invariant.
- **Operational Threshold:** EAR_THRESHOLD = 0.25
- **Temporal Filter:** EAR_CONSEC_FRAMES = 20 (at ~15-20 FPS, this equates to ~1.2 - 1.5 seconds of sustained eyelid closure, cleanly filtering out natural blinks of 150-300ms while immediately catching microsleep and driver sleep episodes).

2. Strict RFID-to-Face Binding Protocol

A critical vulnerability in traditional fleet systems is **credential sharing / buddy punching** (a driver borrowing another person's RFID keycard or vehicle token). DriveSafe solves this through an unbreakable **Hardware-Software Mutual Interlock**.

Step	Entity	Action & Security Assertion
1. Card Tap	ESP32 Edge	Driver taps physical RFID card. ESP32 checks UID in local registry. Crucially: Does NOT start driving session. Enters pendingVerification = true with a 60-second countdown.
2. OLED Alert	SSD1306 HUD	OLED displays: RFID: [Name] → >> VERIFY FACE << → On Website Now! with live countdown timer. Driver cannot bypass this screen.
3. Camera Scan	Web Frontend (/verify)	Driver positions face in frame and triggers Capture & Verify . Video snapshot is POSTed to Flask backend /verify.
4. Vector Match & UID Retrieval	Flask Vision Server	Extracts 128-D embedding. Matches face against faces.pkl. Retrieves registered name AND registered RFID UID for that individual face.
5. Device Handshake	Flask → ESP32 REST	Backend POSTs {"name": "...", "rfid": "..."} directly to http://[ESP32_IP]/verify.
6. Dual-Condition on Gate	ESP32 Firmware	ESP32 asserts: (pendingVerification == true) && (pendingRfidUid == received_rfid) && (pendingDriver->name == received_name). • MATCH: startDriver() invoked, green LED ON, session started. • MISMATCH: Rejects with HTTP 403, triggers siren alarm, displays RFID MISMATCH or FACE MISMATCH.

3. Web Application Modules & UI Functions

The web application is built with a high-performance modern stack (Vite + React 18 + TanStack Router + Tailwind CSS):

- **Live Telemetry HUD (/dashboard):** Consumes live Firebase Realtime Database socket subscriptions. Displays real-time vehicle speed, engine status, continuous shift timer, vehicle battery voltage, GSM/Wi-Fi signal RSSI, satellite fix count, and live latitude/longitude with OpenStreetMap coordinates.
- **Identity Verification Portal (/verify):** Accesses local webcam via WebRTC navigator.mediaDevices.getUserMedia. Renders live camera canvas, captures pristine JPEG frame, handles loading states, displays biometric match confirmation, and shows real-time ESP32 hardware session unlock badge.
- **Driver Enrollment & Provisioning Portal (/admin):** Administrative interface for fleet managers to enroll new drivers. Captures reference portrait, associates driver full name and physical RFID hex UID, and pushes new vector to vision database.
- **Emergency SOS Command:** Renders active vehicle emergency alerts with red visual alarms and enables dispatchers to trigger or acknowledge hardware sirens remotely.

4. Hardware Firmware Edge Capabilities

- **GPS NMEA Stream Parsing:** Hardware UART2 communicates with NEO-6M GPS at 9600 baud. TinyGPS++ parses raw NMEA sentences (\$GPRMC, \$GPGGA) to extract coordinates with 6-decimal precision, speed in km/h, and satellite validity.
- **Fatigue Regulation & Break Bonus:** Enforces maximum continuous driving shift limits. If a driver takes a rest break of ≥ 15 minutes between sessions, the system calculates a rest bonus and credits quota back to their shift allowance.
- **Pre-Alert Warning:** Exactly 10 minutes prior to mandatory shift exhaustion, the ESP32 issues audio beeps and yellow alert prompts on the SSD1306 OLED, prompting the driver to pull over safely.
- **Fail-Safe SOS Interrupt:** Physical push button with hardware debouncing. Holding for 3 seconds engages full emergency mode: alternating siren tones on piezo buzzer, emergency telemetry logged to Firebase, and OLED emergency broadcast.

5. Judge Defense & Viva Voce Q&A; Cheat Sheet

Use these expert answers to confidently answer questions from technical evaluators and judges:

Q1: Why did you use Eye Aspect Ratio (EAR) with landmark regression instead of training a CNN image classifier for drowsiness?

Answer: A 2D CNN classifier for 'open' vs 'closed' eyes requires high computational power, suffers from lighting variations, and struggles with diverse eye shapes. EAR, based on Kazemi & Sullivan's regression trees, runs at 60+ FPS on standard CPU hardware with negligible memory footprint. Furthermore, EAR is mathematically scale-invariant and distance-invariant because the numerator and denominator scale proportionally with distance from the camera.

Q2: How do you distinguish between normal natural blinking and real drowsiness/microsleep?

Answer: Human blinks typically last between 100ms and 300ms (approximately 2 to 6 frames at 20 FPS). In our algorithm, we set `EAR_CONSEC_FRAMES = 20`. The system only triggers a drowsiness state when the EAR remains below the 0.25 threshold for more than 20 consecutive frames (~1.2 to 1.5 seconds of sustained eyelid closure). This completely eliminates false triggers from regular blinks while reliably detecting true microsleep.

Q3: What prevents someone from using an authorized driver's stolen RFID card?

Answer: The strict RFID-to-face binding protocol makes this impossible. Tapping an RFID card only puts the ESP32 into a 'pending verification' state. The system requires facial verification on the camera within 60 seconds. The vision backend matches the face, retrieves the enrolled card UID for THAT specific face, and sends it to the ESP32. If the face doesn't belong to the tapped card's registered owner, the ESP32 rejects the unlock (HTTP 403) and sounds an alarm.

Q4: Why not run the facial recognition and computer vision directly on the ESP32?

Answer: An ESP32 has 520 KB of internal SRAM and a dual-core 240 MHz Tensilica CPU. Running a 3-million-parameter ResNet-34 or a 99.7 MB 68-point regression model requires several hundred megabytes of RAM and vectorized matrix operations. Our distributed architecture places the heavy vision processing on an edge node (in-vehicle compute unit or laptop) while the ESP32 handles real-time sensor polling (GPS, RFID, OLED, Buzzer, Relays) with deterministic, microsecond-level responsiveness.

Q5: What happens if the network or internet connection drops while driving?

Answer: The system is designed for edge resilience. The ESP32's local web server, RFID access control, GPS parsing, OLED display, and piezo buzzer function completely autonomously without internet. Telemetry writes to Firebase are non-blocking HTTP requests. If the internet drops, local driver monitoring and emergency hardware functions continue uninterrupted.

Q6: How does the system handle poor lighting or night-time driving?

Answer: The dlib facial landmark detector operates on grayscale contrast gradients. In low-light automotive environments, standard vehicle interior illumination or near-infrared (NIR) LED illuminators coupled with an IR-sensitive camera (standard in automotive DMS) provide high-contrast landmark localization without dazzling the driver.